If a problem occurs when EF Core tries to interact with your database—you haven't run the migrations to update the database schema, for example—this code throws an exception. I haven't shown it here, but it's important to handle these exceptions in your application so you don't present an ugly error message to user when things go wrong.

Assuming that all goes well, EF Core updates all the autogenerated IDs of your entities (`RecipeId` on `Recipe`, and both `RecipeId` and `IngredientId` on `Ingredient`). Return the recipe ID to the endpoint handler so the handler can use it—to return the ID in the API response, for example.

TIP The `DbContext` type is an implementation of both the unit-of-work and repository patterns, so you generally don't need to implement these patterns manually in your apps. You can read more about these patterns at https://martinfowler.com/eaaCatalog.

And there you have it. You've created your first entity with EF Core. In section 12.4.2 we'll look at loading these entities from the database so you can fetch them all in a list.

12.4.2 Loading a list of records

Now that you can create recipes, you need to write the code to view them. Luckily, loading data is simple in EF Core, relying heavily on LINQ methods to control the fields you need. For your app, you'll create a method on `RecipeService` that returns a summary view of a recipe, consisting of `RecipeId`, `Name`, and `TimeToCook` as a `RecipeSummaryViewModel`, as shown in figure 12.12.